

SAS — Milestones & Templates

Shipment Alerting System · Dart Global Logistics

A deep-dive into how milestone data is modelled, built, synced, and consumed — tables, JSONB, the field registry, milestone library, templates, snapshots, the sync jobs, the REST APIs, and how the Next.js frontend wires into the Flask backend.

Technical reference for project evaluation. Line numbers are approximate and refer to the current codebase.

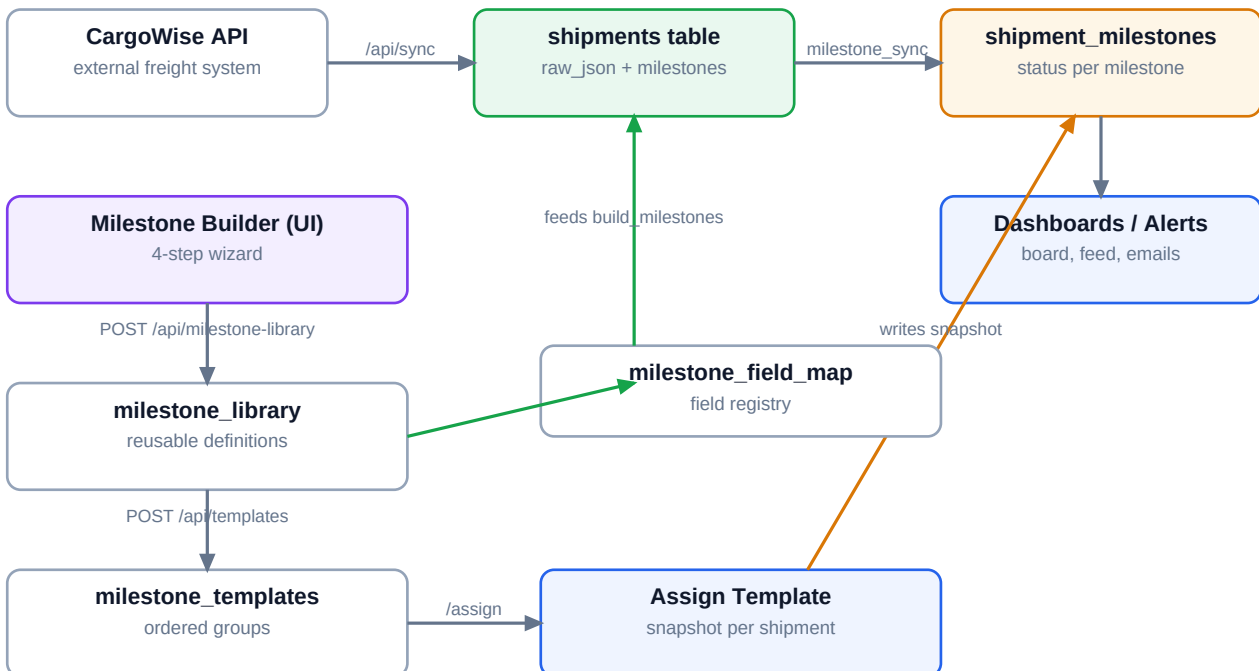
1. The 30-second mental model

Milestones in SAS live in three layers. Keeping them separate is the single most important idea:

- **Definitions (reusable):** what a milestone *is* — its check logic and alert rules. Stored in `milestone_library`, grouped into `milestone_templates`.
- **Assignments (per shipment):** applying a template copies each milestone — *frozen* — into `shipment_milestones` as a **snapshot**. Editing the library later does NOT change already-assigned shipments.
- **Live status:** a background job re-reads each shipment's real data and sets each milestone's status (pending / completed / overdue / delayed). Dashboards and alerts read this.

So: **build a definition** → **group into a template** → **assign to shipments (snapshot)** → **sync recomputes status** → **frontend shows it**.

2. End-to-end workflow



Two engines feed `shipment_milestones`: the **data sync** (top row) keeps raw shipment data fresh; the **assignment** (bottom) attaches definitions. The **status recompute** job combines them into a live status.

3. JSONB — the question you asked

Yes — JSONB is a PostgreSQL column data type that stores a whole JSON object inside a single cell.

Instead of one column per field, one `jsonb` cell holds a nested structure. "B" means *binary*: Postgres parses the JSON on write and stores it in an efficient binary form, so you can query and index inside it — much faster than plain text JSON.

SAS uses JSONB in three places:

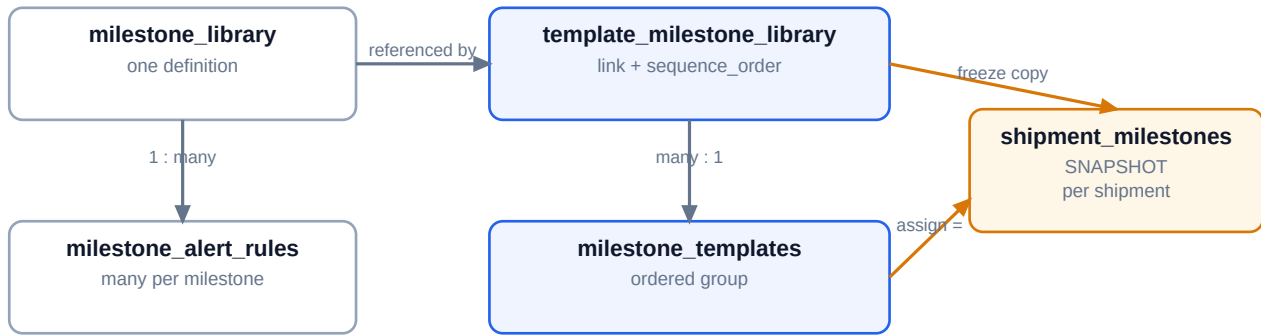
- `shipments.raw_json` — the **entire** original CargoWise record, kept verbatim so nothing is ever lost.
- `shipments.milestones` — a cleaned, grouped summary, e.g. `{"cargo_pickup": {"cargo_pickup_date": "2026-09-02"}}`.
- `shipment_milestones.milestone_snapshot / alert_rules_snapshot` — the frozen copy of a milestone's definition + rules at assignment time.

The snapshot pattern only works because we can freeze an arbitrary definition object into one cell; and `resolve_field_value()` reads a value by looking *inside* the `milestones` JSONB group for the milestone's key.

4. The tables — what each is for

Table	What it holds	Why it exists
<code>shipments</code>	One row per freight job. Mapped columns + <code>raw_json</code> (full original) + <code>milestones</code> (jsonb summary).	The imported copy of CargoWise data. Everything else builds on it.
<code>shipment_milestones</code>	One row per milestone per shipment: <code>status</code> , <code>due_date</code> , <code>completed_date</code> , <code>assigned_email</code> , <code>sequence_order</code> , plus <code>milestone_snapshot / alert_rules_snapshot</code> .	The trackable layer. Dashboards, board, and alerts read this.
<code>milestone_field_map</code>	Registry rows: <code>milestone_key</code> , <code>api_field</code> , <code>canonical_field</code> , <code>is_active</code> .	Maps an incoming CargoWise field to the milestone that consumes it. Handles renames without code.
<code>milestone_library</code>	Reusable milestone definitions: <code>name</code> , <code>milestone_type</code> , <code>check_fields</code> , <code>milestone_key</code> , <code>is_active</code> , <code>is_system_default</code> .	The catalogue of milestones you can drop into any template.
<code>milestone_alert_rules</code>	Alert rules linked to a library milestone (timing, condition, recurrence, recipient, stop).	When/whom to alert for that milestone.
<code>milestone_templates</code>	A named, ordered set of milestones for a shipment type.	The thing you actually assign to shipments.
<code>template_milestone_library</code>	Link rows: which library milestone (or a template-local config) belongs to a template, and its <code>sequence_order</code> .	The many-to-many join between templates and milestones (new model).
<code>template_milestones</code>	Legacy name-only milestone rows for older templates.	Backwards compatibility; new templates use the library links above.

How they relate: a `milestone_library` row has many `milestone_alert_rules`. A `milestone_templates` row has many `template_milestone_library` links, each pointing to one library milestone (or a template-local config). Assigning copies each link's definition + rules into a `shipment_milestones` snapshot.



5. What is_active means (field registry)

Every registry row carries `is_active` (true/false) — a **soft on/off switch**. The sync only reads rows where `is_active = true` (`load_field_map()`, `cargowise_service.py:33`). When a milestone is deleted or a field retired, the code sets `is_active = false` instead of hard-deleting (`deactivate_milestone_fields()`, `field_registry.py:136`). History is preserved, the mapping can be turned back on, and the sync stops collecting a retired field.

The same soft-delete idea appears on `milestone_library.is_active` and `milestone_alert_rules.is_active` — nothing milestone-related is ever truly deleted, so already-assigned shipments keep working.

6. The field registry — how a field reaches a milestone

Register (when a milestone is built)

`register_milestone_fields()` (`field_registry.py:118`) upserts one row per field the milestone uses, so the next sync starts collecting it. Called from `create_library_milestone()`.

Write (during CargoWise sync)

```
build_milestones(item, field_map)          # cargowise_service.py:166
{ key: { canonical_field: item[api_field] } for each mapped field }
```

Values are pulled under `api_field` but stored under `canonical_field` (falls back to `api_field` when null) — how a renamed field is routed back to the name the milestone expects.

Read (when status is computed)

`resolve_field_value(shipment, milestone_key, api_field)` (`field_registry.py:66`): exact key in the milestone's JSONB group → normalized/alias match → plain shipment column.

7. Building a milestone (the library)

Frontend: the 4-step wizard under `components/milestones/MilestoneBuilder/`:

Step / file	What it captures
Step1_BasicInfo.jsx	Name, description, criticality, <code>milestone_type</code> (date / document / comparison / status / missing / custom).
Step2_FieldLinking.jsx	Which field(s) the check reads, and the due-date basis (another field / days after creation / after previous / manual).
Step3_AlertRules.jsx	Alert rules: timing, condition (always / if_not_recorded / if_comparison_true / if_missing), recurrence, recipient, stop conditions.
Step4_Preview.jsx	Read-only review before saving.
MilestoneBuilderShell.jsx	Holds wizard state and step navigation.

Where used: create page `admin/milestone_library/create/page.jsx` POSTs the assembled object to `/api/milestone-library` (~32). Edit page `admin/milestone_library/[id]/page.jsx` loads it (~76), PUTs changes (~92), and reads `/usage` (~77) to warn how many templates use it.

Backend: `routes/milestone_library.py`:

API	Function : line	Does
GET <code>/api/milestone-library</code>	<code>get_all_library_milestones</code> : 59	List all definitions (+ rules).
GET <code>.../<id></code>	<code>get_library_milestone</code> : 91	One definition with rules.

API	Function : line	Does
POST /api/milestone-library	create_library_milestone : 135	Insert definition + alert rules; register its fields. Admin only.
PUT .../<id>	update_library_milestone : 253	Edit definition, replace rules. Does NOT touch assigned shipments.
DELETE .../<id>	delete_library_milestone : 358	Soft-deactivate (is_active=false).
POST .../<id>/duplicate	duplicate_library_milestone : 408	Clone a definition.
GET .../<id>/usage	get_milestone_usage : 461	Which templates reference it.
POST .../promote	promote_local_milestone : 541	Turn a template-local milestone into a reusable library one.

A definition's shape: name, milestone_type, primary_field, expected_date_source/field/offset, tracking_field, field_a/operator/field_b/fixed_value/threshold_value, extra_logics[], logic_combine, milestone_key, plus an alert_rules[] array — each rule inserted into milestone_alert_rules.

8. Building & editing templates

A template is an ordered set of milestones for a shipment type. Milestones come either from the library (source: 'library') or as a template-local config (source: 'local').

Frontend:

Page / component	API it calls (approx line)
admin/milestone_templates_list/page.jsx	GET /api/templates (56) — the list.
components/milestones/Templatestable.jsx	GET /api/templates/<id>/shipments (34,187); DELETE /api/templates/<id> (311).
admin/milestone_template_create/page.jsx	POST /api/templates (62 draft, 92 publish).
admin/milestone_template/page.jsx	GET (107); PUT (158); POST /copy (193); DELETE (223); GET /shipments (126); POST /promote (181).
components/milestones/TemplateMilestoneBuilder.jsx	GET /api/milestone-library (92); POST /api/milestone-library (166); slots → template payload (355).
components/milestones/AssignTemplateModal.jsx	GET /api/templates/<id> (191); GET /preview-assignment (288,317); POST /assign (336).

Backend: routes/templates.py:

API	Function	Does
GET /api/templates	get_all_templates	List active templates (+ counts).
GET .../<id>	get_template	One template with linked milestones + rules.
POST /api/templates	create_template	Create a template and its milestone links.
PUT .../<id>	update_template	Replace name + milestone links.
POST .../<id>/copy	copy_template	Save as a new copy (uses current editor state if sent).
DELETE .../<id>	delete_template	Soft-deactivate.
GET .../preview-assignment	preview_assignment	Which shipments match a mode/client/branch filter.
POST .../<id>/assign	assign_template_to_shipments	The snapshot step (below).
GET .../<id>/shipments	get_template_shipments	Shipments currently using this template.

9. Assignment — where the snapshot is created

`assign_template_to_shipments()` (templates.py) is the heart of the system. For each selected shipment it:

- resolves the template's milestones (`_resolve_template_milestones`);
- on conflict, **skips** or **replaces** (deletes existing `shipment_milestones` first);
- builds one row per milestone via `_snapshot_row()` — copying the definition into `milestone_snapshot` and the rules into `alert_rules_snapshot`, computing an initial `due_date`;
- applies admin-chosen manual/field/relative due-date bases at assign time (`_resolve_manual`).

Why snapshot? so a shipment's tracking never changes underneath it when someone edits the library or template later.

10. The syncs and the scheduler

Two data jobs:

- **CargoWise** → **shipments**: `run_sync()` (routes/sync.py / app.py). Trigger with `POST /api/sync`.
- **shipments** → **shipment_milestones**: `run_milestone_sync()` (sync/milestone_sync.py) derives the default milestones.
- **status recompute**: `recompute_milestone_status()` (services/status_recompute.py) sets completed / overdue / delayed / pending.

The scheduler lives in `app.py` (~line 335), APScheduler + CronTrigger, timezone Asia/Colombo:

Job	Schedule	app.py line
Fixed CargoWise sync	<code>hour='0,6,12,18'</code>	340
Custom extra syncs	loaded from <code>sync_schedules</code> table	349
Field-mismatch check	<code>minute=15 (hourly)</code>	364
Milestone status recompute	<code>minute='*/30'</code>	372
Field-watch scan	<code>minute='5,35'</code>	380
Digest emails (sales/ops/super)	<code>hour=8, minute=0</code>	388-404

Changing the sync time

Edit the `CronTrigger(hour='0,6,12,18', minute=0)` on **app.py line 340**. To add extra times without code, insert a row in `sync_schedules` via the UI (`POST /api/sync/schedules`); the startup loop at **app.py:349** registers one scheduler job per row — that is what "one job per row, so several custom times can coexist" means.

`sync/database_sync.py`'s `SYNC_INTERVAL_MINUTES=30` is a **separate standalone runner**, not what the Flask app uses. The live schedule is the `app.py` cron block.

11. How status becomes what the user sees

`recompute_milestone_status()` evaluates each frozen snapshot check against the shipment's current data:

- **completed** — the check is satisfied;
- **overdue** — outstanding and its resolved deadline has passed;
- **delayed** — outstanding, no passed deadline, but a *later* milestone already completed (out of sequence);
- **pending** — outstanding, no evidence it is late.

The frontend reads these: the **Current Milestones board** (`MilestoneBoard.tsx` → `/api/shipments/all-milestones`), the **alert feed** (`AlertFeedTable.tsx` → `/api/alerts/active`), and the per-shipment modal. All are scoped by role server-side (`services/scope.py`).

12. How the frontend connects to the backend

Every page is a Next.js (App Router) client component. It calls Flask with `fetch()` against `http://127.0.0.1:5000`, attaching a bearer token from `localStorage` via an `authHeaders()` helper. Flask (`CORS(app)`) returns JSON, stored in React state and rendered.

```
const res = await fetch(`${API}/api/milestone-library`, { headers: authHeaders() });
const json = await res.json();
setMilestones(json.data); // React state -> UI
```

The browser never touches the database directly — it only talks to Flask routes, and Flask is the only thing that talks to Supabase/Postgres. That boundary is why auth and role-scoping are enforced in one place.

13. Quick file map

Concern	Backend	Frontend
Milestone definitions	routes/milestone_library.py	admin/milestone_library/*, MilestoneBuilder/*
Templates	routes/templates.py	admin/milestone_template*, TemplateMilestoneBuilder.jsx, Templatestable.jsx
Assign	routes/templates.py (assign)	components/milestones/AssignTemplate Modal.jsx
Field registry	routes/field_map.py, services/field_registry.py	admin/field_registry/page.jsx
CargoWise sync	routes/sync.py, services/cargowise_service.py	admin/system_settings
Milestone sync	sync/milestone_sync.py, sync/database_sync.py	-
Status recompute	services/status_recompute.py	(board / feed)
Scheduler	app.py (~335)	-
Role scoping	services/scope.py	(server-side)

End of reference. Ask for the next area (alert engine, role scoping, or the emailing/digest pipeline) and it can be documented the same way.